Cosmos EVM 개발 및 배포 가이드

1. 개발 환경 설정

1.1 필수 요구사항

- **Go**: 1.23.8 이상
- **Node.js**: 16.0 이상 (스마트 컨트랙트 개발용)
- **Git**: 버전 관리
- **Make**: 빌드 도구

1.2 저장소 클론 및 빌드

```
# 저장소 클론
git clone https://github.com/cosmos/evm.git
cd evm

# 의존성 설치
go mod download

# 바이너리 빌드
make build
```

```
# 테스트 실행
make test
```

1.3 로컬 개발 네트워크 실행

```
# 로컬 노드 시작 (기존 데이터 삭제)
./local_node.sh -y

# 기존 데이터 유지하며 시작
./local_node.sh -n

# 디버깅 모드로 시작
./local_node.sh --remote-debugging
```

2. 체인 커스터마이징

2.1 기본 설정 수정

```
// evmd/config/config.go
const (
// 체인 ID 설정
ChainID = "mychain_1234-1"
// 기본 토큰 denomination
BaseDenom = "amytoken"
```

```
// 가스 한도
DefaultGasLimit = 30_000_000
)
```

2.2 Genesis 파일 커스터마이징

```
{
"app_name": "evmd",
"app_version": "v1.0.0",
"genesis_time": "2024-01-01T00:00:00Z",
"chain_id": "mychain_1234-1",
"app_state": {
"evm": {
"params": {
"evm_denom": "amytoken",
"enable_create": true,
"enable_call": true,
"extra_eips": [],
"chain_config": {
"homestead_block": "0",
"dao_fork_block": "0",
"eip150_block": "0",
"eip155_block": "0",
```

```
"eip158_block": "0",
"byzantium_block": "0",
"constantinople_block": "0",
"petersburg_block": "0",
"istanbul_block": "0",
"berlin_block": "0",
"london_block": "0"
}
}
},
"feemarket": {
"params": {
"no_base_fee": false,
"base_fee_change_denominator": 8,
"elasticity_multiplier": 2,
"enable_height": "0",
"base_fee": "1000000000"
}
}
}
}
```


2.3 커스텀 프리컴파일 추가

```go
// precompiles/custom/my_precompile.go
package custom

import (
"github.com/ethereum/go-ethereum/common"
"github.com/ethereum/go-ethereum/core/vm"
)

type MyPrecompile struct {
keeper MyKeeper
}

func (p *MyPrecompile) Address() common.Address {
return  common.HexToAddress("0x0000000000000000000000000000000000000500")
}

func (p *MyPrecompile) RequiredGas(input []byte) uint64 {
return  1000  // 기본 가스 비용
}

func (p *MyPrecompile) Run(evm *vm.EVM, contract *vm.Contract, readonly  bool) (
// 커스텀 로직 구현
```

```
method, args, err := p.parseInput(contract.Input)
if err != nil {
return nil, err
}
switch method.Name {
case "myFunction":
return p.myFunction(evm, args)
default:
return nil, fmt.Errorf("unknown method: %s", method.Name)
}
}
```

3. 스마트 컨트랙트 개발

3.1 개발 환경 설정

```
# Hardhat 프로젝트 초기화
mkdir my-contracts
cd my-contracts
npm init -y
npm install --save-dev hardhat @nomiclabs/hardhat-ethers ethers

# Hardhat 설정
```

```
npx hardhat
```

3.2 Hardhat 설정 파일

```
// hardhat.config.js
require("@nomiclabs/hardhat-ethers");

module.exports = {
solidity: "0.8.19",
networks: {
cosmosevm: {
url: "http://localhost:8545",
chainId: 262144,
accounts: [
// 테스트 계정 프라이빗 키
"0x..."
]
}
}
};
```

3.3 Cosmos 통합 컨트랙트 예시

```
// contracts/CosmosIntegration.sol

pragma solidity ^0.8.19;


interface IBank {

function balanceOf(address account, string memory denom) external view returns (

function transfer(address to, string memory denom, uint256 amount) external retu

}


interface IStaking {

function delegate(string memory validatorAddr, uint256 amount) external returns

function undelegate(string memory validatorAddr, uint256 amount) external return

}


contract CosmosIntegration {

address constant BANK_PRECOMPILE = 0x0000000000000000000000000000000000000400;

address constant STAKING_PRECOMPILE = 0x0000000000000000000000000000000000000401

event TokensStaked(address indexed user, string validator, uint256 amount);

event TokensTransferred(address indexed from, address indexed to, uint256 amount

function stakeTokens(string memory validatorAddr, uint256 amount) external {

// 스테이킹 프리컴파일 호출

(bool success,) = STAKING_PRECOMPILE.call(

abi.encodeWithSignature("delegate(string,uint256)", validatorAddr, amount)
```

```
);

require(success, "Staking failed");

emit TokensStaked(msg.sender, validatorAddr, amount);

}

function transferCosmosTokens(address to, string memory denom, uint256 amount) e

// 뱅크 프리컴파일 호출

(bool success,) = BANK_PRECOMPILE.call(

abi.encodeWithSignature("transfer(address,string,uint256)", to, denom, amount)

);

require(success, "Transfer failed");

emit TokensTransferred(msg.sender, to, amount);

}

function getCosmosBalance(address account, string memory denom) external view re

(bool success, bytes memory data) = BANK_PRECOMPILE.staticcall(

abi.encodeWithSignature("balanceOf(address,string)", account, denom)

);

require(success, "Balance query failed");

return abi.decode(data, (uint256));

}

}
```

3.4 배포 스크립트

```
// scripts/deploy.js
const { ethers } =  require("hardhat");


async  function  main() {
// 컨트랙트 팩토리 가져오기
const  CosmosIntegration  =  await  ethers.getContractFactory("CosmosIntegration
// 컨트랙트 배포
const  contract  =  await  CosmosIntegration.deploy();
await  contract.deployed();
console.log("CosmosIntegration deployed to:", contract.address);
// 초기 설정
const  tx  =  await  contract.initialize();
await  tx.wait();
console.log("Contract initialized");
}


main()
.then(() =>  process.exit(0))
.catch((error) => {
console.error(error);
process.exit(1);
```

```
});
```

4. 테스트 및 디버깅

4.1 단위 테스트

```
// test/CosmosIntegration.test.js
const { expect } = require("chai");
const { ethers } = require("hardhat");

describe("CosmosIntegration", function () {
let contract;
let owner;
beforeEach(async function () {
[owner] = await ethers.getSigners();
const CosmosIntegration = await ethers.getContractFactory("CosmosIntegration
contract = await CosmosIntegration.deploy();
await contract.deployed();
});
it("Should get Cosmos balance", async function () {
const balance = await contract.getCosmosBalance(
owner.address,
"atest"
```

```
);
expect(balance).to.be.gt(0);
});
it("Should transfer Cosmos tokens", async function () {
const recipient = "0x742d35Cc6634C0532925a3b8D4C9db96590c6C87";
const amount = ethers.utils.parseEther("1.0");
await expect(
contract.transferCosmosTokens(recipient, "atest", amount)
).to.emit(contract, "TokensTransferred");
});
});
```

4.2 통합 테스트

```
# Hardhat 테스트 실행
npx hardhat test --network cosmosevm

# 특정 테스트 파일 실행
npx hardhat test test/CosmosIntegration.test.js --network cosmosevm

# 가스 리포트 생성
npx hardhat test --gas-reporter
```

4.3 디버깅 도구

```
// 트랜잭션 트레이싱
const tx = await contract.myFunction();
const trace = await network.provider.send("debug_traceTransaction", [
tx.hash,
{ tracer: "callTracer" }
]);
console.log(JSON.stringify(trace, null, 2));

// 상태 덤프
const state = await network.provider.send("debug_dumpBlock", ["latest"]);
console.log(state);
```

5. 프로덕션 배포

5.1 바이너리 빌드

```
# 프로덕션 빌드
make build-linux

# 정적 링크 빌드
CGO_ENABLED=0 GOOS=linux go build -a -ldflags '-extldflags "-static"' -o
```

```
# Docker 이미지 빌드
docker build -t my-cosmos-evm:latest .
```

5.2 Genesis 파일 생성

```
# 키 생성
evmd keys add validator --keyring-backend test

# Genesis 파일 초기화
evmd init mychain --chain-id mychain_1234-1

# 밸리데이터 계정 추가
evmd add-genesis-account $(evmd keys show validator -a --keyring-backend

# Genesis 트랜잭션 생성
evmd gentx validator 1000000000000000000amytoken --chain-id mychain_1234-1

# Genesis 파일 완성
evmd collect-gentxs
```

5.3 설정 파일 최적화

```
# config/app.toml

[api]

enable = true

swagger = true

address = "tcp://0.0.0.0:1317"



[grpc]

enable = true

address = "0.0.0.0:9090"



[json-rpc]

enable = true

address = "0.0.0.0:8545"

ws-address = "0.0.0.0:8546"



[evm]

tracer = "json"

max-tx-gas-wanted = 500000



[feemarket]

enabled = true
```

5.4 시스템 서비스 설정

```
# /etc/systemd/system/evmd.service
[Unit]
Description=Cosmos EVM Node
After=network.target

[Service]
Type=simple
User=cosmos
WorkingDirectory=/home/cosmos
ExecStart=/usr/local/bin/evmd start
Restart=on-failure
RestartSec=3
LimitNOFILE=4096

[Install]
WantedBy=multi-user.target
```

6. 모니터링 및 유지보수

6.1 메트릭 수집

```
// 커스텀 메트릭 추가
var (
```

```
evmTxCounter = prometheus.NewCounterVec(
prometheus.CounterOpts{
Name: "evm_transactions_total",
Help: "Total number of EVM transactions",
},
[]string{"status"},
)
gasUsedHistogram = prometheus.NewHistogram(
prometheus.HistogramOpts{
Name: "evm_gas_used",
Help: "Gas used by EVM transactions",
},
)
)
```

6.2 로그 설정

```
# config/config.toml
[log]
level = "info"
format = "json"

[instrumentation]
```

```
prometheus = true

prometheus_listen_addr = ":26660"
```

6.3 백업 및 복구

```
# 상태 백업
evmd export > genesis_backup.json

# 데이터 디렉토리 백업
tar -czf data_backup.tar.gz ~/.evmd/data

# 복구
evmd unsafe-reset-all
cp genesis_backup.json ~/.evmd/config/genesis.json
tar -xzf data_backup.tar.gz -C ~/
```

7. 보안 고려사항

7.1 키 관리

- **하드웨어 지갑**: 프로덕션에서는 하드웨어 지갑 사용
- **키 로테이션**: 정기적인 키 교체
- **다중 서명**: 중요한 작업에 다중 서명 적용

7.2 네트워크 보안

- **방화벽**: 필요한 포트만 개방
- **TLS**: HTTPS/WSS 사용
- **DDoS 방어**: 레이트 리미팅 및 DDoS 방어

7.3 스마트 컨트랙트 보안

- **감사**: 컨트랙트 배포 전 보안 감사
- **업그레이드 가능성**: 프록시 패턴 사용
- **권한 관리**: 적절한 접근 제어

이 문서는 Cosmos EVM의 개발부터 배포까지의 전체 과정을 상세히 설명합니다. 마지막 문서에서는 고급 기능과 최적화 방법을 살펴보겠습니다.